Trees and Ensembles — Key Concepts

Lesson 7 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

13 November 2026 · a one-page index

Contents

A single tree 1
Averaging trees 2
Boosting 2
Choosing 3

An index, not a summary. Each entry says what a term means and where the handout treats it properly. Use this to find your way back into the text, or to check before an exercise that there is no word here you could not define.

A single tree

Decision tree. A sequence of yes/no questions chosen one at a time, each splitting a group into two purer ones, until a stopping rule fires. → § 2.1

Greedy splitting. At every step the tree takes whichever split helps most *right now*, with no way to look ahead to one that would help more two levels down. → § 2.1

Gini impurity. The probability that two examples drawn at random from a group, labelled by the group's own class frequencies, would disagree. Zero when the group is pure. → § 2.2

Impurity reduction. What a candidate split is worth: the parent's impurity minus the children's, weighted by how the group was divided. → § 2.2

Why $1 - \sum p_c^2$ counts disagreements. Two draws agree on class c with probability p_c^2; what is left over is the chance they disagree. → § 2.2

Depth as the bias-variance dial. Shallow describes only the coarsest structure; deep enough isolates single points, and at full depth the training accuracy is exactly 1 unless two examples share every feature value but not the label. → § 3

A depth cap against a leaf budget. `max_depth` caps every root-to-leaf path and splits every node above the cap that can be split; `max_leaf_nodes` gives the whole tree a budget of leaves and spends it best-first, on whichever leaf offers most, which is the better knob for a tree meant to be read. → § 4

Instability. An early split reroutes everything beneath it, so a small change in the data changes the tree's shape. The strongest split survives resampling; the weaker ones, which are most of the

tree, do not. → § 4

Averaging trees

Bootstrap sample. m rows drawn with replacement from m rows, so some appear more than once and some not at all. → § 5

Bagging. Fit a tree to each bootstrap sample and take the majority vote. The tree algorithm is untouched; only what surrounds it changes. scikit-learn averages the trees' probabilities instead, which for fully grown trees is the same thing. → § 5

Out-of-bag rows. The rows a given bootstrap sample never drew — free validation data for the trees that did not see them. → § 5.1

Out-of-bag score. Each row scored only by the trees whose bootstrap sample missed it — a different third of the forest for each row. It works like cross-validation without an explicit split, estimates the same quantity, and agrees within its own noise. → § 5.1

The variance floor. Averaging drives variance down towards the trees' shared correlation times their individual variance — not to zero. Past that point, more trees buy nothing. → § 5.2

What bagging cannot fix. An error every tree shares. Averaging removes the part of the trees' mistakes that differs from tree to tree; trees too shallow for the rule stay too shallow, however many are averaged. → § 5.2

Random forest. Bagging plus one restriction: at each split only a random subset of features is offered as a candidate, which forces the trees to disagree and lowers the floor. → § 6

Feature importance. The total impurity reduction each feature is credited with, summed over every split and tree. Suggestive, not definitive. → § 7

Why importance misleads with many irrelevant columns. Two causes. The restriction that decorrelates the trees also leaves the real features off the menu at most splits; and deep in a tree, on a handful of examples, some noise column separates the group a little better by chance. → § 7

Permutation importance. The check on it: shuffle one column of held-out data and measure how much accuracy drops. A noise column costs almost nothing, up to the chance of the shuffle. → § 7.1

Boosting

Boosting. Building trees *in sequence*, each fitted specifically to correct what the ensemble so far got wrong. → § 8

Weak learner. The shallow tree — depth two or three — that each round adds. → § 8

Regression tree. The same greedy algorithm, fit to numbers rather than classes: a leaf predicts the mean of its values, a split is scored by squared error. → § 8

Residual. For squared error, what remains unexplained: the truth minus the current prediction. → § 8

Functional gradient descent. The general recipe the residual is a special case of: each tree approximates the negative gradient of the loss with respect to the ensemble's current output. → § 8

Pseudo-residual. That negative gradient when the loss is not squared error. For log-loss it is the true label minus the predicted probability — the same quantity Lesson 4's logistic regression descended against. → § 9

Learning rate in boosting. How much of each new tree's correction is actually applied. It trades off against the number of trees: both control how much total correction the ensemble makes. → § 10

Early stopping. Stop adding trees once the score on held-out training data has stopped improving. One of two ways to choose the number of trees, the other being to cross-validate it; one of them is needed, because boosting keeps reducing its training loss for as long as it runs. → § 10

Choosing

Ensembling beats the choice of ensemble, on this dataset. Every ensembling strategy here lands close to the others and all of them closer to the ceiling than any single tree — so here *whether* you ensemble matters far more than *how*. The exercise shows a dataset where it does not. → § 11

Flexibility is not an advantage on its own. The unconstrained tree is the worst model on the leaderboard despite being the most flexible: flexibility without a stopping rule is a larger space to get lost in. → § 11

When a single tree is still the right answer. When the decision has to be explained to the person it affects, a depth-tuned tree is the only model here a person can read start to finish. → § 11